

保障LLM自动生成动态SQL性能&效率全套落地方案

适用系统：Java增强RAG项目（Ollama+Chroma+SQLite结构化库）

适用模块：SqlGenerator LLM自动生成SQL模块

一、整体五层性能管控架构

从生成源头→代码校验→数据库执行→并发调度→缓存兜底五层闭环，杜绝全表扫描、慢SQL、数据库雪崩问题：

1. Prompt源头约束层：强制LLM输出高性能SQL语法
2. 代码拦截校验层：自动拦截低效违规SQL，直接降级
3. 数据库优化层：索引、超时、只读账号底层保障
4. 并发调度层：并行执行+超时熔断控制耗时
5. 多级缓存层：缓存SQL生成结果、DB查询数据减少重复开销

二、第一层：Prompt源头强约束（最高优先级）

修改SqlGenerator内SQL生成提示词，新增硬性性能规则，让大模型从根源写出高效SQL。

完整新增性能约束片段（直接追加到原有Prompt顶部）

- # 【强制性能硬性规则，违反则输出无效SQL】
1. 禁止使用 `SELECT *`，必须只写出用户问题明确需要的字段；
 2. 必须携带WHERE索引字段过滤条件，禁止无过滤全表查询；
 3. `LIKE`模糊查询仅允许后缀匹配 `关键词%`，禁止前后全模糊 `%关键词%`；
 4. 所有SQL强制携带`LIMIT`，取值范围5~10，最大不超过20；
 5. 禁止嵌套子查询、超过2张表JOIN关联；
 6. 无排序需求禁止写`ORDER BY`，排序仅允许索引字段；
 7. 聚合`COUNT/SUM`必须搭配WHERE过滤，禁止统计全表；
 8. 禁止函数包裹索引列（例：`WHERE UPPER(name)=?` 会失效索引）；
 9. 优先等值匹配查询，减少模糊检索使用；

优劣SQL示例对比

低效违规SQL（会被拦截）

```
SELECT * FROM sdk_info limit 10
```

标准高性能合规SQL

```
SELECT name,support_format FROM sdk_info WHERE name = ? limit 10
```

三、第二层：代码层性能校验拦截（必落地）

扩展原有securityCheck方法，新增低效SQL拦截逻辑，识别到全表、全模糊、SELECT * 等语句直接放弃DB查询，自动降级仅使用向量文档回答。

完整Java校验代码

```
public boolean securityCheck(String sql) {
    String lower = sql.toLowerCase();
    // 1. 原有危险操作拦截（删改语句）
    String[] blacklist = {"drop", "delete", "update", "alter", "insert", "truncate"};
    for (String word : blacklist) {
        if (lower.contains(word)) return false;
    }
    // 2. 基础安全校验（参数占位+强制分页）
    if (!lower.contains("limit")) return false;
    if (!lower.contains("?")) return false;

    // =====新增性能拦截规则=====
    // 禁止SELECT * 全字段查询
    if (lower.contains("select *")) return false;
    // 禁止前后全包模糊匹配
    if (lower.matches(".*like '%.*%'")) return false;
    // 禁止无WHERE条件全表扫描
    if (!lower.contains("where")) return false;
    // 禁止超过2表JOIN复杂关联
    if (lower.split("join").length > 2) return false;
    // 无索引字段排序拦截（根据自身索引调整字段）
    if (lower.contains("order by") && !lower.contains("name") && !lower.contains("stars"))
        return false;

    return true;
}
```

拦截后降级逻辑：校验返回false → 跳过SQL执行流程，dbData置空，仅使用向量检索文档生成答案。

四、第三层：数据库底层性能优化

4.1 高频查询字段建立索引（核心提速手段）

以项目sdk_info业务表为例，所有LLM常用WHERE过滤字段创建索引：

```
-- SDK名称（最高频查询条件）
CREATE INDEX idx_sdk_name ON sdk_info(name);
-- 开发语言过滤索引
CREATE INDEX idx_sdk_lang ON sdk_info(lang);
-- 支持文件格式索引
CREATE INDEX idx_sdk_format ON sdk_info(support_format);
```

4.2 数据库连接超时限制

SQLite连接设置查询超时，长时间未返回自动终止，卡死线程：

```
// SqlExecutor 获取连接时配置
conn.setQueryTimeout(1); // 单位：秒，超过1秒强制终止SQL
```

4.3 账号权限隔离兜底

- 业务数据库账号仅授予SELECT只读权限，杜绝数据篡改；
- 禁止业务账号DDL建表、删表权限。

4.4 大数据量分页规则

数据量超万行时，在Prompt追加强制分页规则，禁止一次性拉取大量数据集。

五、第四层：并发执行与超时熔断优化

5.1 向量检索 & SQL查询并行执行

原RAG流水线串行执行，改造异步并发，缩短整体响应耗时：

```
// 异步并行两个检索任务
CompletableFuture> vecFuture = CompletableFuture.supplyAsync(() -> {
    // 多路向量召回逻辑
});
CompletableFuture>> sqlFuture = CompletableFuture.supplyAsync(() -> {
    // SQL生成+校验+执行完整逻辑
});
// 等待双任务完成
List uniqueDocs = vecFuture.get();
List> dbData = sqlFuture.get();
```

5.2 SQL查询超时熔断机制

```
CompletableFuture>> sqlTask = CompletableFuture.supplyAsync(() -> {
    return sqlExecutor.execute(sqlResp.sql, sqlResp.params);
});
List> dbData;
try {
    // 1秒超时控制
    dbData = sqlTask.get(1, TimeUnit.SECONDS);
} catch (TimeoutException e) {
    dbData = Collections.emptyList();
    System.out.println("SQL查询超时，跳过结构化数据查询");
}
```

六、第五层：多级缓存优化（大幅减少LLM与DB开销）

6.1 SQL生成缓存（缓存问题→SQL语句+参数）

```
// SqlGenerator 类内静态本地缓存
private static final ConcurrentHashMap sqlGenCache = new ConcurrentHashMap<>();
```

```
public SqlResp generate(String userQuery) throws Exception {  
    // 命中缓存直接返回，跳过LLM调用  
    if (sqlGenCache.containsKey(userQuery)) {  
        return sqlGenCache.get(userQuery);  
    }  
    // 原有LLM生成逻辑省略  
    SqlResp resp = new SqlResp();  
    sqlGenCache.put(userQuery, resp);  
    return resp;  
}
```

6.2 数据库查询结果缓存

- 单机部署：ConcurrentHashMap本地内存缓存；
- 分布式多服务：Redis持久缓存；

效果：重复查询完全不访问数据库，毫秒级返回数据。

七、监控、兜底与持续优化方案

7.1 慢SQL埋点日志

```
long start = System.currentTimeMillis();  
List<T> result = pstmt.executeQueryWrapper();  
long cost = System.currentTimeMillis() - start;  
// 日志埋点  
log.info("自动SQL执行完成, sql={},耗时={}ms", sql, cost);
```

7.2 三级自动降级策略

1. LLM生成SQL存在性能违规 → 跳过DB查询；
2. SQL执行超时/数据库异常 → 跳过DB查询；
3. 无WHERE过滤条件全表查询 → 跳过DB查询；

7.3 持续迭代优化手段

- 收集线上低效SQL案例，作为Few-shot示例写入Prompt；
- 定期更新数据库索引，新增业务字段同步建索引；
- 监控缓存命中率，优化缓存过期策略。

八、业务专属精简优化（适配ag-psd/袜子设计RAG系统）

1. 强制通过name字段精准匹配SDK，杜绝无过滤全表扫描；
2. 禁用%关键词%全模糊匹配，仅允许尾部模糊检索；
3. 高频问题（ag-psd支持格式、开发语言）开启缓存，几乎零DB访问；
4. 表数据量小，LIMIT统一固定为5，最小化数据返回体积。

九、落地执行优先级清单（由易到难）

1. 【必做】Prompt追加性能约束规则；
2. 【必做】扩展securityCheck，拦截低效SQL；

3. 【必做】业务查询字段建立数据库索引，配置SQL超时；
4. 【推荐】两级缓存：SQL生成缓存、DB查询结果缓存；
5. 【推荐】向量检索与SQL查询并行执行；
6. 【进阶】SQL执行耗时埋点、慢查询日志监控。